

Module	Fundamentals of Machine Learning: EEEM066
Assessment type	Coursework assignment
Weighting	20%
Deadline	4:00PM, Tue, 2 nd December 2025 (Week 11)
STUDENT URN	6940610
STUDENT Name	Anil Emanuel

Objectives:

- Learn and become familiar with the process of sequential hyperparameter tuning of an entire deep learning pipeline.
- Develop intuition via experimentation on how to use such model design and parameter tuning to improve the performance of a pipeline.
- Interpret experimental results of deep learning experiments and their implications.

Outcomes:

- Navigate general deep learning codebases and modify according to requirements.
- Empirically suggest reasonable values for hyperparameters and select the ones enabling the best performance.
- Understand the implications of changing the backbone architecture of a pipeline.
- Reason why data augmentation works.
- Understand the implications and importance of selecting the best learning rate and batch size.

- Understand how to report experimental results professionally. This includes using tables, plots, and diagrams to communicate ideas and results.

Knife classification in real-world images

For this coursework assignment, you are encouraged to apply what you have learnt from the module materials and your participation in collaborative learning activities like class discussions.

Guidelines

- Submit your assignment as a **Word or PDF document**, along with the supporting evidence as required, via the SurreyLearn submission page before the deadline. Do not submit any other unrelated documents that could flood your submission and mislead the marking process. Only the latest version submitted before the deadline will be assessed if multiple versions exist.
- Label your submission file as: EEEM066-[insert your URN number]-Assignment e.g., EEEM066-1234567-Assignment
- Include a reference list at the end of your assignment, and use the [Harvard](#) or [IEEE](#) referencing style when citing sources. Please use the selected option consistently.
- **Word limit:** Each section of the report should not exceed **200 words** (excluding tables, plots, and graphs). Penalties of up to 50% may be applied for unnecessarily long reports.
- This assignment requires time and effort. It is recommended that you start working on it early to minimise the possibility of experiencing stress around it and avoid missing the deadline.
- You will receive both an overall mark and written feedback on your assignment.
- Extensions for this assignment will only be granted under exceptional circumstances. Extension requests must be submitted via the Extenuating Circumstances (ECs) process, as tutors are not permitted to approve ad hoc extension requests.

Note:

Please follow Surrey's guidelines for [avoiding plagiarism](#).

You must complete this coursework individually. Copying code or text from the internet or another student and including it in your assignment without proper attribution constitutes plagiarism.

Undetected plagiarism undermines the quality of your degree by hindering the assessor's ability to evaluate your work fairly and prevents you from learning through the coursework. If plagiarism is suspected, you will be referred to an Academic Misconduct Panel, which may result in academic sanctions.

All assignments will be checked for plagiarism using Turnitin. Therefore, it is important that you correctly reference sources that have informed your work where appropriate.

Task

Review the instructions for the assignment carefully before starting. This assignment is marked out of 100 and will contribute **20%** to your total module grade. The three sections of the assignment are worth **90 marks**, with an additional **10 marks** allocated to the presentation and clarity of the report.

Background

In recent years, the number of crimes involving sharp objects, such as knives, has increased all across the world. In the year ending March 2022, England and Wales saw around 45,000 offences involving a knife or sharp instrument. This was 9% higher than in 2020/21 and 34% higher than in 2010/11.

Therefore, there is a need to develop an AI-based weapon analysis system that enables officers to identify the make, model, and seller of knives using photographs. Although functional, current knife recognition and labelling methods are beset by inefficiencies due to strained public resources, the possibility of human error, and the UK's expanding digital footprint.

The following figure provides a visual representation of a knife classification system.

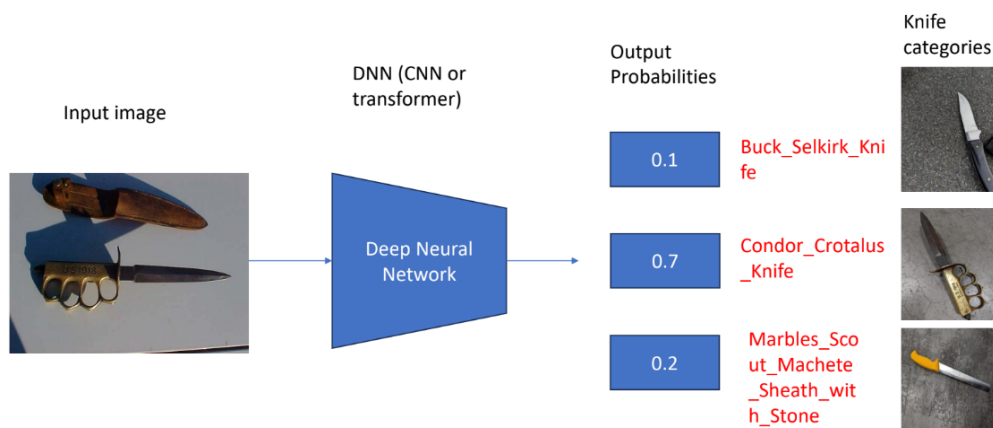


Figure 1: Overview of a knife classification system.

Instructions

In this assignment, you will develop a deep-learning-based system to classify knife images into different categories i.e., classes. You will gain real-world experience in designing, training, and evaluating a deep neural network for accurate, fine-grained knife classification.

You are required to document your findings in a report, divided into three sections as outlined later. Complete each section (in this document) according to the provided guidance. ***The results should be for the test set*** (not for the train/validation set).

Dataset

Use the “Knife classification” training dataset (provided [here](#)), which includes approximately 25,250 knife images categorised into 543 classes, along with a validation dataset of 440 images and a test dataset consisting of 469 images to complete the assignment. The performance of your model will be evaluated on test dataset using the mean Average Precision (mAP) metric.

Important note:

Please do not distribute this dataset or use it for any purposes other than this assignment.

Compute platform

To complete this coursework assignment, you will need to use Google Colab. You are welcome to revisit the lab sheets you engaged with during this module for further guidance that can support you in completing this assignment.

Getting started

To prepare for this assignment, it is recommended that you engage with the following readings, available via the Reading List, before starting work on developing your knife classification system:

- “Fine-grained image analysis with deep learning: A survey” by Wei Xiu-Shen, Jianxin Wu, and Quan Cui
- “Weapon classification using deep convolutional neural network” by Neelam Dwivedi, Dushyant Kumar Singh, and Dharmender Singh Kushwaha

Additionally, you are encouraged to revisit the lectures in this module and the lab sessions on using Python and PyTorch to train and test CNNs and Vision transformers for image classification. The PyTorch Introductory lab sheet and the lecture slides on these topics are available on SurreyLearn.

Support in programming

You can consult this [GitHub repository](#) for a reference code base. Should you experience any problems during the development of your knife classification, please feel free to open an issue.

Deliverables

For this assignment, you are required to run your project in the Colab. For the report, you must document the experiments performed using your software.

Additional instructions

Keep the following guidance in mind as you work on producing the deliverables for this assignment:

Hyperparameters

For the parameters that are not learnable, you can set them before starting the training.

Format

- Ensure your responses to each question are on the spot as shown in Figure 2. Focus on the specific question and not move away.

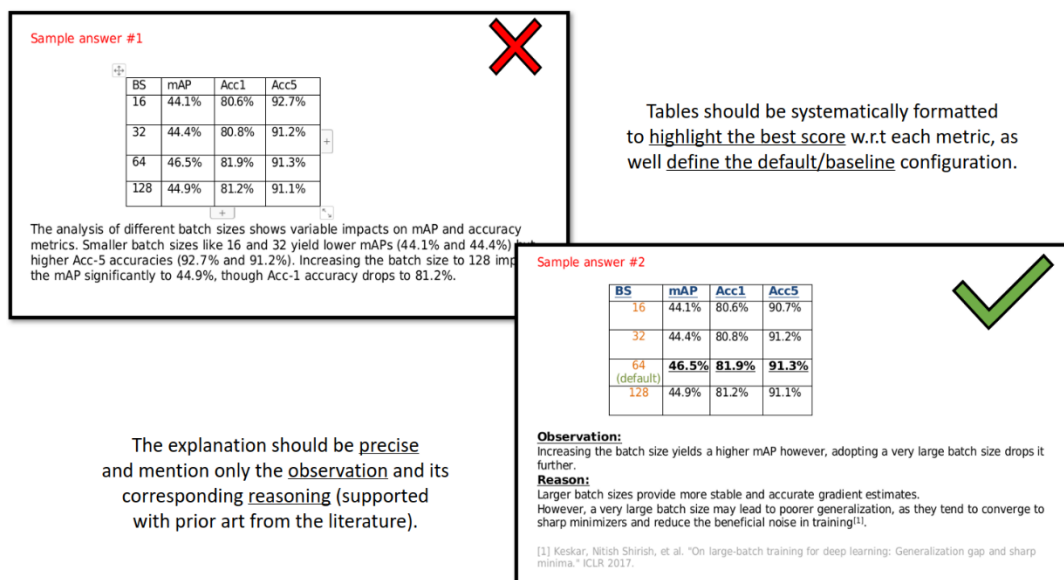


Figure 2: Response example.

Log files

- Submit your own log files from your codebase. You are required to submit a separate log file for each experiment-based question.
- Do not change the log file structure. Each log file is watermarked and unique to each run.
- Name each log file according to the section number and question number: (log_sec{Section_num}_q{Question_num}.txt). For example, the log file generated for an experiment that corresponds to Section 1 Question 2, should be named as "log_sec1_q2.txt".

Presentation and clarity

In addition to the technical content, the presentation and clarity of your writing will be assessed.

Model performance metrics

While different metrics are available, it is recommended that you prioritise mAP when selecting the best-performance models.

Start your assignment

Write your assignment in the designated space provided below. Each of the sections outlines the tasks required and specifies the information you must include in your report for that section.

Section 1: Familiarity with the code provided (40 marks)

1. Run the code using the default settings. Discuss the steps involved in the training and evaluation process and explain the impact of the observed performance using an appropriate metric. (20 marks)
2. Apply a different CNN variant from the one used in the code provided. Critically compare the results with those observed in Question 1. (10 marks)
3. Apply one more neural network architecture (CNN or Vision transformer) different from those in Questions 1 and 2. Critically discuss and compare the results from these previous two steps. (10 marks)

Note:

For Questions 2 and 3, ensure the new architecture is explicitly specified in your shell script command if you are using one from the codebase.

Start writing here:

1.1 Baseline Model – EfficientNet-B0

The performance of the default code was carried out with EfficientNet-B0, with colour-jitter augmentation, Adam optimisation, and validation after each epoch. It yielded a test mAP of **0.449**, showing that it had a rather moderate capacity for fine-grained knife category recognition. While compound scaling makes EfficientNet-B0 efficient, its lightweight design limits feature extraction for subtle class differences. This baseline performance serves as a reference for further architectural comparisons.

Table 1. EfficientNet-B0 Baseline Performance

Model	Test mAP
EfficientNet-B0	0.449

1.2 Alternative CNN – ResNet-50

The ResNet-50 was a deeper classic CNN, introduced with residual connections for improved gradient flow. It had achieved a slightly lower test mAP of **0.444** compared to EfficientNet-B0, despite its depth. This could be from the older architectural design of ResNet-50 being less able to capture the fine texture and shape variations required in this dataset.

Table 2. ResNet-50 Performance

Model	Test mAP
ResNet-50	0.444

1.3 Modern Architecture – ConvNext-Tiny

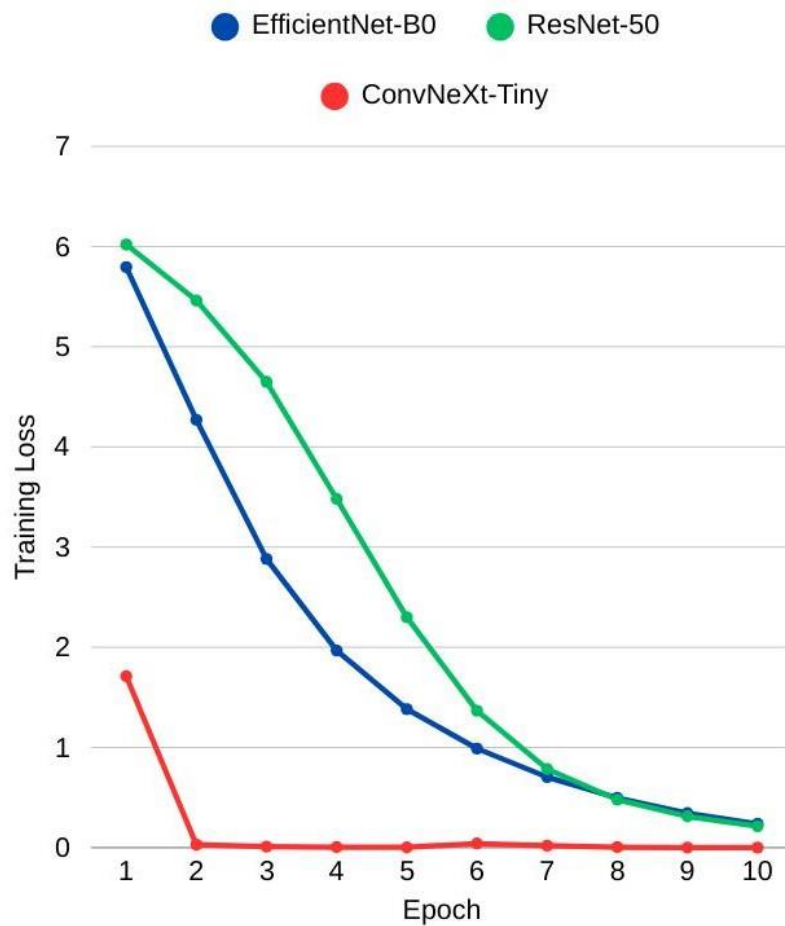
ConvNext-Tiny represented a state-of-the-art CNN, which was inspired by the Vision Transformers design. Usage of large-kernel depthwise convolutions and LayerNorm enabled strong spatial modeling. So it achieved a test mAP of 0.678, outperforming the previous two architectures.

Table 3. ConvNext-Tiny Performance

Model	Test mAP
ConvNext-Tiny	0.678

Table 4. Model Performance in Section 1

Model	Section	Test mAP
EfficientNet-B0	Q1	0.449
ResNet50	Q2	0.444
ConvNext-Tiny	Q3	0.678



Line Graph Explanation (Section 1):

ConvNext-Tiny converges faster than EfficientNet-B0 and ResNet-50, It only took a few epochs to reaches near-zero loss in the training-loss curves.

Section 2: Dataset preparation and augmentation experiment (30 marks)

Begin with the default data augmentation setting of the default command line, which uses colour jittering.

1. Further append on top two additional data augmentation techniques, one at a time e.g., Default + “random rotation”, Default + “horizontal flip”. Compare the results with the default configuration in the provided code and discuss the differences in performance. (20 marks)
2. Combine the augmentation techniques from Question 1 to find the best-performing combination. Highlight any improvement or drop in the overall score. (10 marks)

Start writing here:

2.1 Individual Augmentations (Rotation and Horizontal Flip)

This section evaluates additional augmentations using ConvNext-Tiny, the strongest model from Section 1 (baseline test mAP = **0.678**). ConvNext-Tiny already captures fine-grained knife features effectively.

Random Rotation (15°) in Experiment 1, reduced performance to **0.648 mAP**. Although rotation can help some models handle varied orientations, ConvNext-Tiny already exhibits rotational robustness, and additional rotation may slightly distort blade geometry.

Horizontal flip ($p=0.5$) in Experiment 2, resulted in **0.643 mAP**. Many knife classes in the dataset are asymmetric, so flipping can invert important class-specific cues, leading to misclassifications.

Overall, **both augmentation didn't exceeded the baseline**, illustrating that strong, modern architectures may benefit less from generic augmentations when dealing with fine-grained tasks.

Table 1. Individual Augmentation Results

Augmentation	Test mAP
Rotation 15°	0.648
Horizontal Flip 0.5	0.643

2.2 Combined Augmentation (Rotation + Flip)

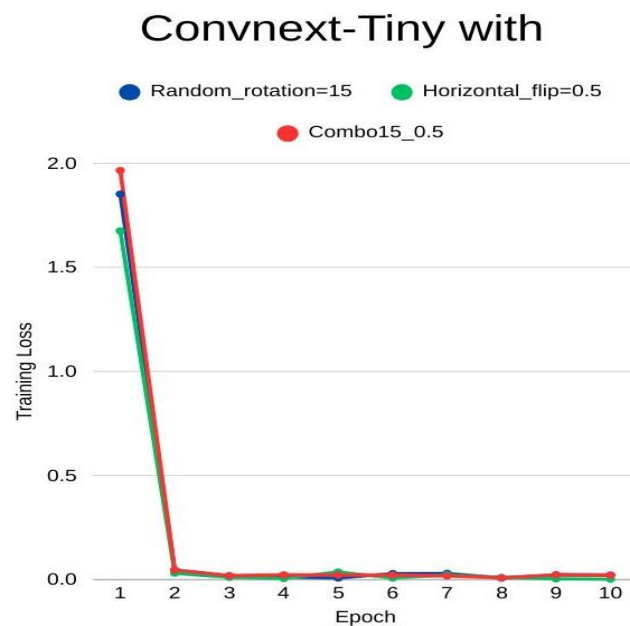
When rotation and horizontal flip were applied together, performance dropped further to **0.570 mAP**. This indicates that combining multiple transformations introduces excessive variation, which can distort the fine-grained visual cues that distinguish knife classes. Since knife classification depends heavily on subtle differences in blade shape, handle design, and edge curvature, strong augmentations can make images diverge too far from the true class distribution.

Table 2. Combined Augmentation

Augmentation	Test mAP
Rotation + Flip	0.570

Table 3. Data Augmentation Results

Augmentation Setting	Description	Test mAP
Default + Rotation (15°)	Colour Jitter + Random Rotation	0.648
Default + Horizontal Flip (0.5)	Colour Jitter + Flip	0.643
Rotation + Flip	Combined Augmentation	0.570



Line Graph Explanation (Section 2):

The training-loss curves show that all three augmentation settings converge extremely quickly, confirming that ConvNeXt-Tiny learns the dataset efficiently.

Section 3: Exploration of Hyperparameters (20 marks)

Start with the default learning rate (LR) and batch size (BS).

1. Exploration of LR. (10 marks)
 - a. Experiment with three values of LR (in addition to the default value).
 - b. Discuss the observed impact of each value on overall performance.
2. Exploration BS. (10 marks)
 - a. Using the best LR value from the experiments in Question 1, test three different values of the BS in addition to the default value.
 - b. Discuss how the changes in BS affect overall performance.

Start writing here:

This section 3 will further investigate the effect of learning rate (LR) and batch size (BS) using the same best model ConvNext-Tiny from section 1.

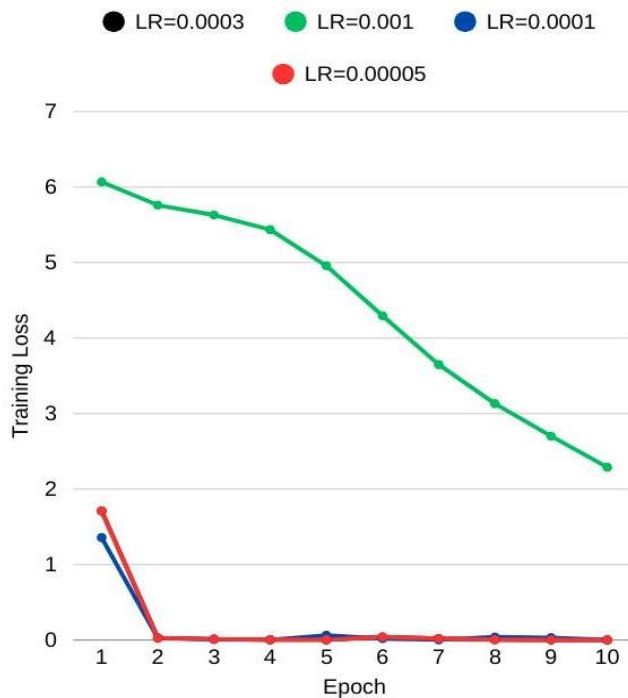
3.1 Learning Rate Experiments

Three learning rates were evaluated: **0.001**, **0.0001**, and **0.00005**. when LR was increased to **0.001** it caused unstable optimisation, leading to a very bad performance collapse (test mAP = **0.043**) due to overly aggressive weight updates. When the LR was reduced to **0.0001** it improved stability and achieved **0.641 mAP**, but was below the baseline. When the LR was further reduced to **0.00005**, it achieved **0.678 mAP**, same as the original baseline. Now these results give clear explanation that ConvNeXt-Tiny benefits from **smaller learning rates**.

Table 1 Experiments Results

Three LR values were tested in addition to the default (0.0003):

Experiment	Learning Rate	Test mAP
LR-0 (Baseline)	0.0003	0.678
LR-1	0.001	0.043
LR-2	0.0001	0.641
LR-3	0.00005	0.678



Line Graph Explanation (Section 3.1):

This illustrates that **LR=0.001 diverges**, decreasing very slowly across epochs, in the training-loss curves. While lower learning rates (0.0003, 0.0001, 0.00005) converge almost immediately, as seen in the graph.

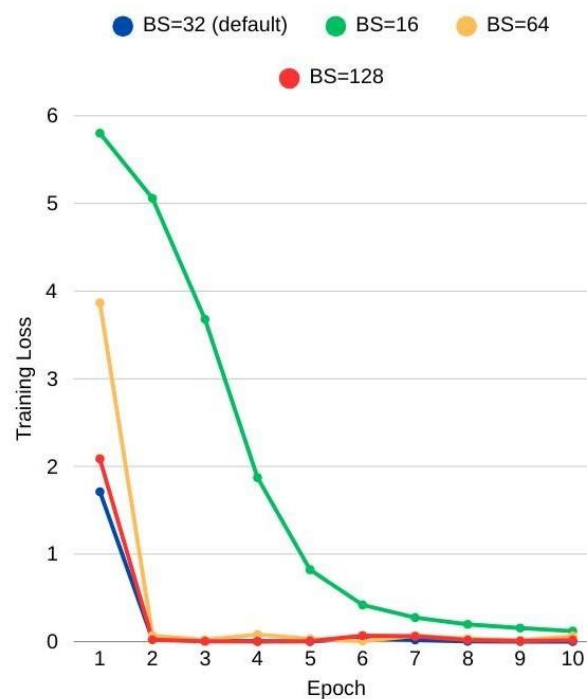
3.2 Batch Size Experiments

Three batch sizes—**16**, **64**, and **128**—were tested by using the best LR (0.0003), when applied a small BS of **16** resulted in noisy gradients and overfitting, reflected low mAP of **0.137**. Another BS of **64** were applied so improved stability but reached only **0.409**, while a larger BS of **128** achieved **0.521**, still below baseline. These findings showed that the original **BS = 32** provides the optimal balance between gradient stability, memory efficiency, and generalisation for ConvNeXt-Tiny on this dataset.

Table 2. Experiment Results

Using the best LR (0.0003), three batch sizes were tested:

Experiment	Batch Size	Test mAP
BS-0 (Baseline)	32	0.678
BS-1	16	0.137
BS-2	64	0.409
BS-3	128	0.521



Line Graph Explanation (Section 3.2):

The default BS=32 in the training-loss curves show that converges quickly and stably, while on the other hand BS=16 begins with the highest loss and learns slowest, and BS=64 and BS=128 converge similarly fast.

References

- [1] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *Proc. ICML*, 2019.
- [2] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," *Proc. CVPR*, 2016.
- [3] Z. Liu, H. Mao, C.-Y. Wu *et al.*, "A ConvNet for the 2020s," *Proc. CVPR*, 2022.
- [4] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *Proc. ICLR*, 2019.
- [5] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS*, 2019.
- [6] C. Szegedy *et al.*, "Going Deeper with Convolutions," *Proc. CVPR*, 2015.
- [7] C. Shorten and T. Khoshgoftaar, "A Survey on Image Data Augmentation for Deep Learning," *Journal of Big Data*, vol. 6, 2019.

Rubric

Section 1: Familiarity with the code provided (40 marks)

Task 1.1: Running and understanding the code (20 marks)

- **Understanding the code execution**
 - *Criteria:* Clear understanding and explanation of the code flow and processes.
 - *Indicators:* Detailed description of data loading, model initialisation, training loop, and evaluation steps.

(5 marks)
- **Metric explanation and performance analysis**
 - *Criteria:* Appropriate choice and understanding of the evaluation metric(s) (e.g., accuracy, mAP).
 - *Indicators:* Correct calculation and interpretation of the metric values. Discussion on what these values imply about the model's performance.

(10 marks)
- **Discussion of observed performance**
 - *Criteria:* Insightful discussion on the observed results.
 - *Indicators:* Analysis of performance patterns, potential overfitting/underfitting issues, and suggestions for improvement.

(5 marks)

Task 1.2: Applying a different CNN variant (10 marks)

- **Implementation of a new CNN variant**
 - *Criteria:* Correctly modifying the code to implement a different CNN architecture.
 - *Indicators:* Model architecture is different from the provided one, and code runs without errors.

(4 marks)
- **Comparison and critical analysis**
 - *Criteria:* Critical comparison between the original and new CNN results.

- *Indicators:* Use of quantitative results to support comparison, discussion on potential reasons for performance differences.

(6 marks)

Task 1.3: Applying another neural network architecture (10 marks)

- **Implementation of a new neural network**

- *Criteria:* Correctly implementing a neural network architecture different from those in Tasks 1.1 and 1.2.
- *Indicators:* Model architecture is distinct, and code runs correctly.

(4 marks)

- **Comprehensive comparison and analysis**

- *Criteria:* Deep comparative analysis between all three architectures.
- *Indicators:* Insightful discussion on the strengths and weaknesses of each model, based on quantitative results and qualitative observations.

(6 marks)

Section 2: Dataset preparation and augmentation experiment (30 marks)

Task 2.1: Augmentation techniques (20 marks)

- **Implementation of additional augmentations**

- *Criteria:* Successful addition of two new augmentation techniques.
- *Indicators:* Correct application and explanation of chosen techniques, code runs without errors.

(8 marks)

- **Performance comparison and analysis**

- *Criteria:* Comparative analysis of the impact of each augmentation setting.
- *Indicators:* Clear explanation of performance differences between default and new settings, supported by quantitative results.

(8 marks)

- **Discussion on results and insights**

- *Criteria:* Insightful interpretation of how augmentations affected model performance.

- *Indicators*: Discussion on potential reasons for performance changes and implications for model robustness.

(4 marks)

Task 2.2: Best-performing augmentation combination (10 marks)

- **Combining techniques and experimentation**
 - *Criteria*: Correct implementation of combined augmentation techniques.
 - *Indicators*: Code successfully combines the techniques and executes without errors.
- **Analysis of improvement or decline**
 - *Criteria*: Detailed discussion on whether the combined augmentation improved performance.
 - *Indicators*: Use of appropriate metrics to highlight any performance change, supported by logical reasoning.

(5 marks)

(5 marks)

Section 3: Exploration of hyperparameters (20 marks)

Task 3.1: Learning rate exploration (10 marks)

- **Experimentation with different LR values**
 - *Criteria*: Correct experimentation with three additional LR values.
 - *Indicators*: Clear documentation of the chosen LR values and implementation.
- **Performance analysis of each LR**
 - *Criteria*: Thorough analysis of the effect of each LR value on performance.
 - *Indicators*: Use of metrics to compare performance across LR values, discussion on the reasons behind observed changes.

(4 marks)

(6 marks)

Task 3.2: Batch size exploration (10 marks)

- **Experimentation with different BS values**
 - *Criteria*: Correct experimentation with three additional BS values.

- *Indicators*: Clear documentation of the chosen BS values and implementation using the best-performing LR from Task 3.1.

(4 marks)

- **Performance analysis of each BS**

- *Criteria*: Detailed analysis of the impact of each BS value on performance.
- *Indicators*: Use of metrics to compare performance across BS values, discussion on the reasons behind observed changes.

(6 marks)

Report writing and presentation (10 marks)

- **Figures should be well-presented, with readable axes and labels.**

(2 marks)

- **The writing should be clear, grammatically correct, and easy to follow.**

(6 marks)

- **Tables are used when discussing results involving multiple hyperparameters values**

- *Criteria*: Clarity, structure, and depth of the written report.
- *Indicators*: Logical flow, clear explanation of methods and results, proper use of figures/tables to support the analysis.

(2 marks)

Total: 100 marks